

## Exceções e controle de erros

### Exemplos de exceções

Alguns possíveis motivos externos para ocorrer uma exceção são:

- Tentar abrir um arquivo que não existe.
- Tentar fazer consulta a um banco de dados que não está disponível.
- Tentar escrever algo em um arquivo sobre o qual não se tem permissão de escrita.
- Tentar conectar em servidor inexistente.

Alguns possíveis erros de lógica para ocorrer uma exceção são:

- Tentar manipular um objeto que está com o valor nulo.
- Dividir um número por zero.
- Tentar manipular um tipo de dado como se fosse outro.
- Tentar utilizar um método ou classe não existentes

Voltando ao exemplo das contas do banco o que aconteceria ao tentar chamar o método `saca` com um valor fora do limite? O sistema mostraria uma mensagem de erro, mas quem chamou o método `saca` não saberá que isso aconteceu.

Como avisar aquele que chamou o método de que ele não conseguiu fazer aquilo que deveria?

Em Java, os métodos dizem qual o contrato que eles devem seguir. Se, ao tentar sacar, ele não consegue fazer o que deveria, ele precisa, ao menos, avisar ao usuário que o saque não foi feito.

Veja no exemplo abaixo: estamos forçando uma Conta a ter um valor negativo, isto é, estar num estado inconsistente de acordo com a nossa modelagem.

```
Conta minhaConta = new Conta();
minhaConta.deposita(100);
minhaConta.setLimite(100);
minhaConta.saca(1000);
// o saldo é -900? É 100? É 0? A chamada ao método saca funcionou?
```

Em sistemas de verdade, é muito comum que quem saiba tratar o erro é aquele que chamou o método e não a própria classe! Portanto, nada mais natural do que a classe sinalizar que um erro ocorreu.

A solução mais simples utilizada antigamente é a de marcar o retorno de um método como boolean e retornar `true`, se tudo ocorreu da maneira planejada, ou `false`, caso contrário:

```
boolean saca(double quantidade) {
    // posso sacar até saldo+limite
    if (quantidade > this.saldo + this.limite) {
```

```

        System.out.println("Não posso sacar fora do limite!");
        return false;
    } else {
        this.saldo = this.saldo - quantidade;
        return true;
    }
}

```

Um novo exemplo de chamada ao método acima:

```

Conta minhaConta = new Conta();
minhaConta.deposita(100);
minhaConta.setLimite(100);
if (!minhaConta.saca(1000)) {
    System.out.println("Não saquei");
}

```

Repare que tivemos de lembrar de testar o retorno do método, mas não somos obrigados a fazer isso. Esquecer de testar o retorno desse método teria consequências drásticas: a máquina de autoatendimento poderia vir a liberar a quantia desejada de dinheiro, mesmo que o sistema não tivesse conseguido efetuar o método saca com sucesso, como no exemplo a seguir:

```

Conta minhaConta = new Conta();
minhaConta.deposita(100);

// ...
double valor = 5000;
minhaConta.saca(valor); // vai retornar false, mas ninguém verifica!
caixaEletronico.emite(valor);

```

Mesmo invocando o método e tratando o retorno de maneira correta, o que faríamos se fosse necessário sinalizar quando o usuário passou um valor negativo como quantidade? Uma solução seria alterar o retorno de boolean para int e retornar o código do erro que ocorreu. Isso é considerado uma má prática (conhecida também como uso de "magic numbers").

Além de você perder o retorno do método, o valor devolvido é "mágico" e só legível perante extensa documentação, além de não obrigar o programador a tratar esse retorno e, no caso de esquecer isso, seu programa continuará rodando já num estado inconsistente.

Repare o que aconteceria se fosse necessário retornar um outro valor. O exemplo abaixo mostra um caso onde, através do retorno, não será possível descobrir se ocorreu um erro ou não, pois o método retorna um cliente.

```

public Cliente procuraCliente(int id) {
    if (idInvalido) {
        // avisa o método que chamou este que ocorreu um erro
    } else {
        Cliente cliente = new Cliente();
        cliente.setId(id);
        // cliente.setNome("nome do cliente");
    }
}

```

```
    return cliente;
}
}
```

Por esses e outros motivos, utilizamos um código diferente em Java para tratar aquilo que chamamos de exceções: os casos onde acontece algo que, normalmente, não iria acontecer. O exemplo do argumento do saque inválido ou do id inválido de um cliente é uma exceção à regra.

### Exceção

Uma exceção representa uma situação que normalmente não ocorre e representa algo de estranho ou inesperado no sistema.

## **Exercício para começar com os conceitos**

Antes de resolvermos o nosso problema, vamos ver como a Java Virtual Machine age ao se deparar com situações inesperadas, como divisão por zero ou acesso a um índice da array que não existe.

1. Para aprendermos os conceitos básicos das exceptions do Java, teste o seguinte código você mesmo:

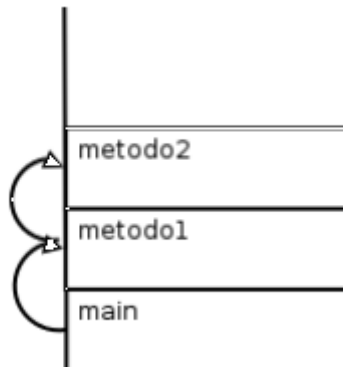
```
class TesteErro {
    public static void main(String[] args) {
        System.out.println("inicio do main");
        metodo1();
        System.out.println("fim do main");
    }

    static void metodo1() {
        System.out.println("inicio do metodo1");
        metodo2();
        System.out.println("fim do metodo1");
    }

    static void metodo2() {
        System.out.println("inicio do metodo2");
        int[] array = new int[10];
        for (int i = 0; i <= 15; i++) {
            array[i] = i;
            System.out.println(i);
        }
        System.out.println("fim do metodo2");
    }
}
```

Repare o método main chamando metodo1 e esse, por sua vez, chamando o metodo2. Cada um desses métodos pode ter suas próprias variáveis locais, isto é: o metodo1 não enxerga as variáveis declaradas dentro do main e por aí em diante.

Como o Java (e muitas das outras linguagens) faz isso? Toda invocação de método é empilhada em uma estrutura de dados que isola a área de memória de cada um. Quando um método termina (retorna), ele volta para o método que o invocou. Ele descobre isso através da pilha de execução (stack): basta remover o marcador que está no topo da pilha:



Porém, o nosso metodo2 propositadamente possui um enorme problema: está acessando um índice de array indevido para esse caso; o índice estará fora dos limites da array quando chegar em 10!

Rode o código. Qual é a saída? O que isso representa? O que ela indica?

```
Console X
<terminated> Teste [Java Application] /caelum/jdk1.5.0_07/bin/java (12/07/2006 4:44:42 PM)
inicio do main
inicio do metodo1
inicio do metodo2
0
1
2
3
4
5
6
7
8
9
Exception in thread "main" java.lang.ArrayIndexOutOfBoundsException: 10
    at Teste.metodo2(Teste.java:18)
    at Teste.metodo1(Teste.java:10)
    at Teste.main(Teste.java:4)
```

Essa é o conhecido rastro da pilha (stacktrace). É uma saída importantíssima para o programador - tanto que, em qualquer fórum ou lista de discussão, é comum os programadores enviarem, juntamente com a descrição do problema, essa stacktrace. Mas por que isso aconteceu?

O sistema de exceções do Java funciona da seguinte maneira: quando uma exceção é lançada (throw), a JVM entra em estado de alerta e vai ver se o método atual toma alguma precaução ao tentar executar esse trecho de código. Como podemos ver, o metodo2 não toma nenhuma medida diferente do que vimos até agora.

Como o metodo2 não está tratando esse problema, a JVM pára a execução dele anormalmente, sem esperar ele terminar, e volta um stackframe pra baixo, onde será feita nova verificação: "o metodo1 está se precavendo de um problema chamado ArrayIndexOutOfBoundsException?" "Não..." Volta para o main, onde também não há proteção, então a JVM morre (na verdade, quem morre é apenas a Thread corrente, veremos mais para frente).

Obviamente, aqui estamos forçando esse caso e não faria sentido tomarmos cuidado com ele. É fácil arrumar um problema desses: basta percorrermos a array no máximo até o seu length.

## TRY / CATCH

```
try
{
    //trecho de código que pode vir a lançar uma exceção
}
catch(tipo_excecao_1 e)
{
    //ação a ser tomada
}
catch(tipo_excecao_2 e)
{
    //ação a ser tomada
}
catch(tipo_excecao_n e)
{
    //ação a ser tomada
}
```

Onde:

- try{ ... } - Neste bloco são introduzidas todas as linhas de código que podem vir a lançar uma exceção.
- catch(tipo\_excessao e) { ... } - Neste bloco é descrita a ação que ocorrerá quando a exceção for capturada.

Porém, apenas para entender o controle de fluxo de uma Exception, vamos colocar o código que vai tentar (try) executar o bloco perigoso e, caso o problema seja do tipo ArrayIndexOutOfBoundsException, ele será pego (caught). Repare que é interessante que cada exceção no Java tenha um tipo... ela pode ter atributos e métodos.

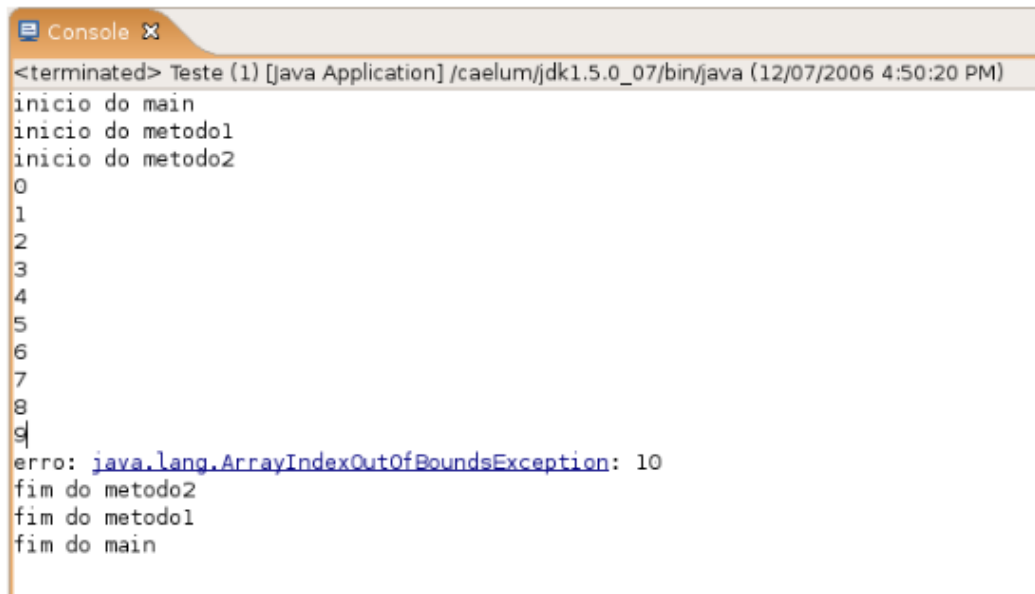
2. Adicione um try/catch em volta do for, pegando ArrayIndexOutOfBoundsException. O que o código imprime?

```
try {
    for (int i = 0; i <= 15; i++) {
        array[i] = i;
        System.out.println(i);
    }
}
```

```

} catch (ArrayIndexOutOfBoundsException e) {
    System.out.println("erro: " + e);
}

```



```

<terminated> Teste (1) [Java Application] /caelum/jdk1.5.0_07/bin/java (12/07/2006 4:50:20 PM)
inicio do main
inicio do metodo1
inicio do metodo2
0
1
2
3
4
5
6
7
8
9
erro: java.lang.ArrayIndexOutOfBoundsException: 10
fim do metodo2
fim do metodo1
fim do main

```

3. Em vez de fazer o try em torno do for inteiro, tente apenas com o bloco de dentro do for:

```

for (int i = 0; i <= 15; i++) {
    try {
        array[i] = i;
        System.out.println(i);
    } catch (ArrayIndexOutOfBoundsException e) {
        System.out.println("erro: " + e);
    }
}

```

Qual é a diferença?



```

<terminated> Teste (1) [Java Application] /caelum/jdk1.5.0_07/bin/java (12/07/2006 4:50:20 PM)
inicio do main
inicio do metodo1
inicio do metodo2
0
1
2
3
4
5
6
7
8
9
erro: java.lang.ArrayIndexOutOfBoundsException: 10
erro: java.lang.ArrayIndexOutOfBoundsException: 11
erro: java.lang.ArrayIndexOutOfBoundsException: 12
erro: java.lang.ArrayIndexOutOfBoundsException: 13
erro: java.lang.ArrayIndexOutOfBoundsException: 14
erro: java.lang.ArrayIndexOutOfBoundsException: 15
fim do metodo2
fim do metodo1
fim do main

```

4. Retire o try/catch e coloque ele em volta da chamada do metodo2.

```

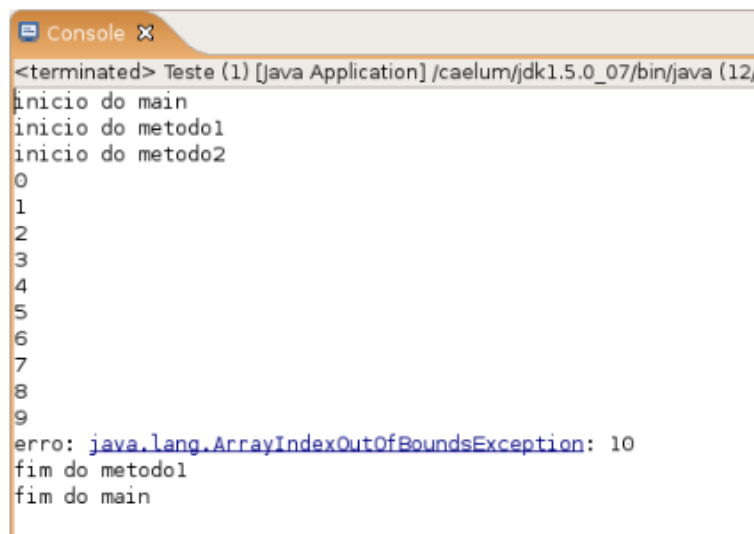
System.out.println("inicio do metodo1");
try {
    metodo2();
}

```

```

} catch (ArrayIndexOutOfBoundsException e) {
    System.out.println("erro: " + e);
}
System.out.println("fim do metodo1");

```



```

<terminated> Teste (1) [Java Application] /caelum/jdk1.5.0_07/bin/java (12/
início do main
início do metodo1
início do metodo2
0
1
2
3
4
5
6
7
8
9
erro: java.lang.ArrayIndexOutOfBoundsException: 10
fim do metodo1
fim do main

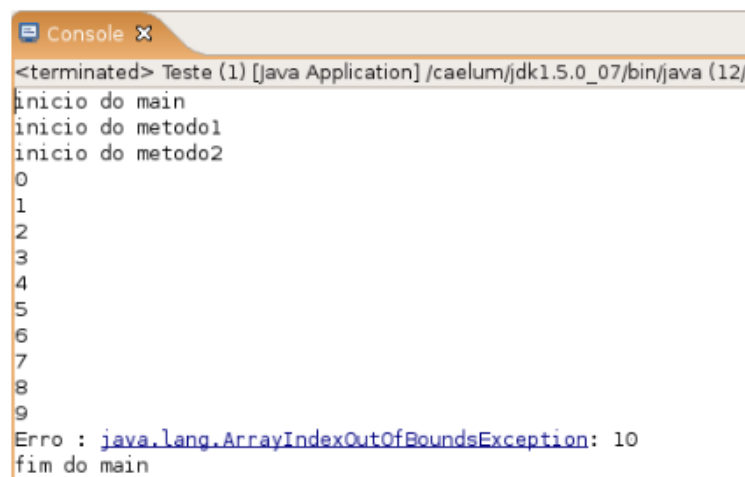
```

5. Faça o mesmo, retirando o try/catch novamente e colocando em volta da chamada do metodo1. Rode os códigos, o que acontece?

```

System.out.println("início do main");
try {
    metodo1();
} catch (ArrayIndexOutOfBoundsException e) {
    System.out.println("Erro : "+e);
}
System.out.println("fim do main");

```



```

<terminated> Teste (1) [Java Application] /caelum/jdk1.5.0_07/bin/java (12/
início do main
início do metodo1
início do metodo2
0
1
2
3
4
5
6
7
8
9
Erro : java.lang.ArrayIndexOutOfBoundsException: 10
fim do main

```

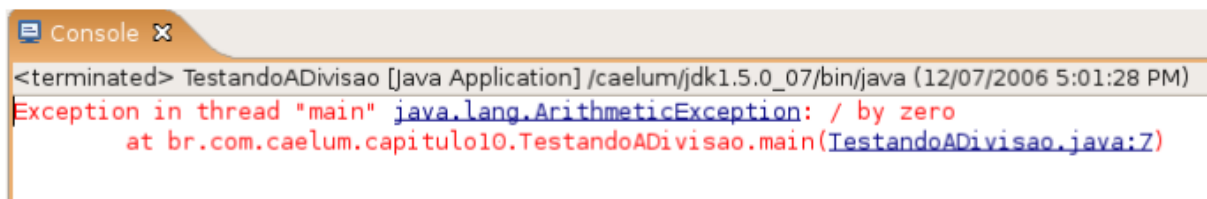
Repare que, a partir do momento que uma exception foi caught (pega, tratada, handled), a execução volta ao normal a partir daquele ponto.

## Exceções de Runtime mais comuns

Que tal tentar dividir um número por zero? Será que a JVM consegue fazer aquilo que nós definimos que não existe?

```
public class TestandoADivisao {  
  
    public static void main(String args[]) {  
        int i = 5571;  
        i = i / 0;  
        System.out.println("O resultado " + i);  
    }  
}
```

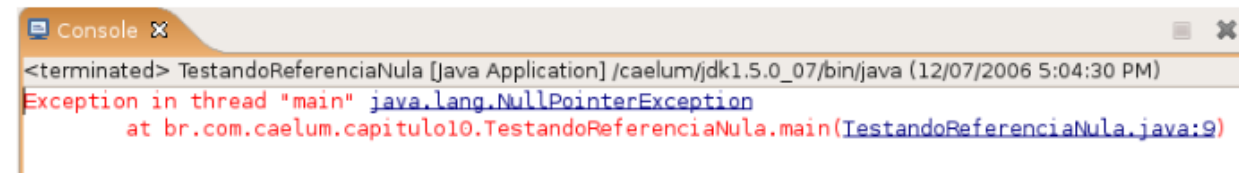
Tente executar o programa acima. O que acontece?



The screenshot shows a console window titled "Console" with the following text:   
<terminated> TestandoADivisao [Java Application] /caelum/jdk1.5.0\_07/bin/java (12/07/2006 5:01:28 PM)  
Exception in thread "main" java.lang.ArithmeticException: / by zero  
at br.com.caelum.capitulo10.TestandoADivisao.main(TestandoADivisao.java:7)

```
public class TestandoReferenciaNula {  
    public static void main(String args[]) {  
        Conta c = null;  
        System.out.println("Saldo atual " + c.getSaldo());  
    }  
}
```

Tente executar este programa. O que acontece?



The screenshot shows a console window titled "Console" with the following text:   
<terminated> TestandoReferenciaNula [Java Application] /caelum/jdk1.5.0\_07/bin/java (12/07/2006 5:04:30 PM)  
Exception in thread "main" java.lang.NullPointerException  
at br.com.caelum.capitulo10.TestandoReferenciaNula.main(TestandoReferenciaNula.java:9)

Repare que um `ArrayIndexOutOfBoundsException` ou um `NullPointerException` poderia ser facilmente evitado com o `for` corretamente escrito ou com `ifs` que checariam os limites da array.

Outro caso em que também ocorre tal tipo de exceção é quando um `cast` errado é feito (veremos mais pra frente). Em todos os casos, tais problemas provavelmente poderiam ser evitados pelo programador. É por esse motivo que o Java não te obriga a dar o `try/catch` nessas exceptions e chamamos essas exceções de `unchecked`. Em outras palavras, o compilador não checa se você está tratando essas exceções.

## Erros

Os erros em Java são um tipo de exceção que também podem ser tratados. Eles representam problemas na máquina virtual e não devem ser tratados em 99% dos casos, já que provavelmente o melhor a se fazer é deixar a JVM encerrar (ou apenas a Thread em questão).

## Outro tipo de exceção: Checked Exceptions

Fica claro, com os exemplos de código acima, que não é necessário declarar que você está tentando fazer algo onde um erro possa ocorrer. Os dois exemplos, com ou sem o try/catch, compilaram e rodaram. Em um, o erro terminou o programa e, no outro, foi possível tratá-lo.

Mas não é só esse tipo de exceção que existe em Java. Um outro tipo, obriga a quem chama o método ou construtor a tratar essa exceção. Chamamos esse tipo de exceção de checked, pois o compilador checará se ela está sendo devidamente tratada, diferente das anteriores, conhecidas como unchecked.

Um exemplo interessante é o de abrir um arquivo para leitura, onde pode ocorrer o erro do arquivo não existir:

```
class Teste {  
    public static void metodo() {  
        new java.io.FileInputStream("arquivo.txt");  
    }  
}
```

O código acima não compila e o compilador avisa que é necessário tratar o FileNotFoundException que pode ocorrer:

```
Teste.java:3: unreported exception java.io.FileNotFoundException; must be caught  
or declared to be thrown  
        new java.io.FileReader("arquivo.txt");  
        ^  
1 error
```

Para compilar e fazer o programa funcionar, temos duas maneiras que podemos tratar o problema. O primeiro, é tratá-lo com o try e catch do mesmo jeito que usamos no exemplo anterior, com uma array:

```
public static void metodo() {  
    try {  
        new java.io.FileInputStream("arquivo.txt");  
    } catch (java.io.FileNotFoundException e) {  
        System.out.println("Nao foi possível abrir o arquivo para leitura");  
    }  
}
```

A segunda forma de tratar esse erro, é delegar ele para quem chamou o nosso método, isto é, passar para a frente.

```
public static void metodo() throws java.io.FileNotFoundException {  
    new java.io.FileInputStream("arquivo.txt");  
}
```

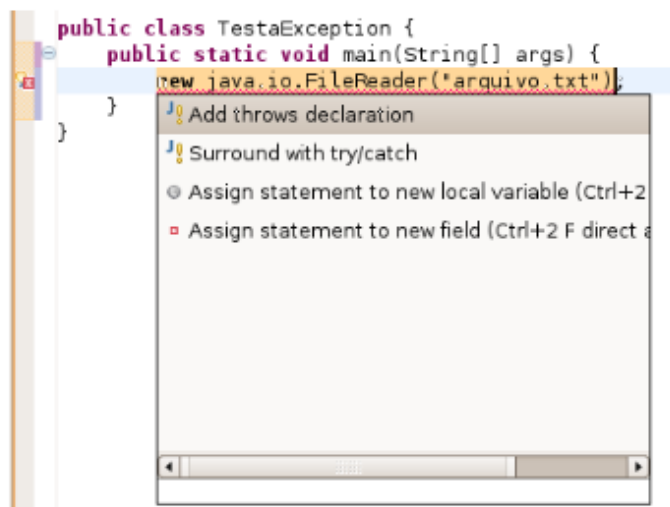
```
}
```

No NetBeans é bem simples fazer tanto um try/catch como um throws:

Tente digitar esse código no NetBeans:

```
public class TestaException {  
    public static void main(String[] args) {  
        new java.io.FileInputStream("arquivo.txt");  
    }  
}
```

O Eclipse vai reclamar :



E você tem duas opções:

1. Add throws declaration, que vai gerar:

```
public class TestaException {  
    public static void main(String[] args) throws FileNotFoundException {  
        new java.io.FileInputStream("arquivo.txt");  
    }  
}
```

2. Surround with try/catch, que vai gerar:

```
public class TestaException2 {  
    public static void main(String[] args) {  
        try {  
            new java.io.FileInputStream("arquivo.txt");  
        } catch (FileNotFoundException e) {  
            // TODO Auto-generated catch block  
            e.printStackTrace();  
        }  
    }  
}
```

```
    }  
  }  
}
```

No início, existe uma grande tentação de sempre passar o problema pra frente para outros o tratarem. Pode ser que faça sentido, dependendo do caso, mas não até o main, por exemplo. Acontece que quem tenta abrir um arquivo sabe como lidar com um problema na leitura. Quem chamou um método no começo do programa pode não saber ou, pior ainda, tentar abrir cinco arquivos diferentes e não saber qual deles teve um problema!

Não há uma regra para decidir em que momento do seu programa você vai tratar determinada exceção. Isso vai depender de em que ponto você tem condições de tomar uma decisão em relação àquele erro. Enquanto não for o momento, você provavelmente vai preferir delegar a responsabilidade para o método que te invocou.

## EXERCÍCIO

Imagine uma classe que tem um método principal **main** que tem como seu único objetivo alterar todas as letras de uma frase para maiúsculas utilizando o método **toUpperCase()** da classe String, caso a frase esteja nula e se tente usar o método `toUpperCase()` na mesma será lançada uma exceção de `NullPointerException`.

Primeiro vamos ver como ficaria a tal classe sem a utilização do **try/catch**.

Exemplo de código sem try-catch

```
public class aumentaFrase {  
    public static void main(String args[])  
    {  
        String frase = null;  
        String novaFrase = null;  
        novaFrase = frase.toUpperCase();  
        System.out.println("Frase antiga: "+frase);  
        System.out.println("Frase nova: "+novaFrase);  
    }  
}
```

Quando este código for executado, o mesmo lançará uma **NullPointerException**, como poder ser visto na saída do console quando executamos tal programa.

### Saída gerada pelo programa sem try-catch

```
Exception in thread "main" java.lang.NullPointerException  
at aumentaFrase.main(aumentaFrase.java:15)
```

Ou seja, o mesmo tentou acessar um atributo de um objeto que estava nulo. Para ajudar a melhorar a situação, deve-se usar o try/catch.

Uma solução do o TRY/CATCH

```
public static void main(String args[])  
{  
    String frase = null;
```

```

String novaFrase = null;
try
{
    novaFrase = frase.toUpperCase();
}
catch(NullPointerException e) //CAPTURA DA POSSÍVEL exceção.
{
    //TRATAMENTO DA exceção
    System.out.println("O frase inicial está nula, para solucionar
tal o problema, foi lhe atribuido um valor default.");
    frase = "Frase vazia";
    novaFrase = frase.toUpperCase();
}
System.out.println("Frase antiga: "+frase);
System.out.println("Frase nova: "+novaFrase);
}

```

Quando este código for executado, o mesmo lançará uma `NullPointerException`, porém esta exceção será tratada desta vez, sendo a mesma capturada pelo **`catch{}`** e dentro deste bloco as devidas providências são tomadas. Neste caso é atribuído um valor default à variável **frase**.